RFC 9110 — HTTP Methods Key Points

Date: 2026-08-02

Source: https://www.rfc-editor.org/rfc/rfc9110.html#name-methods

RFC 9110 (STD 97), HTTP Semantics, June 2022 — Section 9: Methods

9.1 Overview

The request method token is the primary source of request semantics: it indicates the purpose of the request and what the client expects as a successful result. Method tokens are case-sensitive; by convention standardised methods are all-uppercase US-ASCII. Standardised methods are not resource-specific — a method has the same semantics wherever it is applied, though each resource decides whether it implements or allows those semantics.

All general-purpose servers MUST support `GET` and `HEAD`; all other methods are OPTIONAL. An origin server receiving an unrecognised or unimplemented method SHOULD respond `501 (Not Implemented)`. A method that is recognised and implemented but not allowed for the target resource SHOULD yield `405 (Method Not Allowed)`, whose response MUST include an `Allow` header listing the permitted methods.

Method Properties Summary Table

Method	Safe	Idempotent	Cacheable	Request Body	Key Constraint / Note
GET	Yes	Yes	Yes	Should not	Response cacheable; prefer POST when URI exposes sensitive data.
HEAD	Yes	Yes	Yes	Should not	Same headers as GET, never a body; must be supported by all servers.
POST	No	No	Cond.	Yes	Cacheable only with explicit freshness + Content-Location = target URI.
PUT	No	Yes	No	Yes	Creates or replaces resource state; success invalidates cached responses.
DELETE	No	Yes	No	Should not	Removes URI association (not necessarily the data); invalidates cache.
CONNECT	No	No	No	Must not	Target is host:port; port always required; restrict to safe ports.

Method	Safe	Idempotent	Cacheable	Request Body	Key Constraint / Note
OPTIONS	Yes	Yes	No	Optional	Use * for server-wide query; response SHOULD include Allow header.
TRACE	Yes	Yes	No	Must not	Must not carry sensitive data; commonly disabled in production.

9.2 Common Method Properties

9.2.1 Safe Methods

A method is safe if its defined semantics are essentially read-only: the client does not request, and does not expect, any state change on the origin server. **SAFE METHODS: GET, HEAD, OPTIONS, TRACE.** A server may still have side effects (e.g. access logging, ad billing), but the client is not accountable for them. Safety exists so spiders and pre-fetching caches can operate without fear of causing harm. If a resource encodes an unsafe action in query parameters (e.g. 'page?do=delete'), the resource owner MUST disable that action for safe methods.

9.2.2 Idempotent Methods

A method is idempotent if the intended effect of multiple identical requests is the same as that of a single request. **IDEMPOTENT METHODS: PUT, DELETE, plus all safe methods (GET, HEAD, OPTIONS, TRACE).** POST and CONNECT are NOT idempotent. Idempotency enables automatic retry after a communication failure. A client SHOULD NOT auto-retry a non-idempotent method unless it can verify the original was never applied; a proxy MUST NOT auto-retry non-idempotent requests; a client SHOULD NOT auto-retry a failed automatic retry.

9.2.3 Methods and Caching

For a cache to store and reuse a response, the method definition must explicitly allow caching and state the conditions for reuse; a method that does not do so cannot be cached. RFC 9110

defines caching semantics for GET, HEAD, and POST — although the overwhelming majority of cache implementations only support GET and HEAD.

9.3 Method Definitions

GET (Section 9.3.1)

Safe: Yes | **Idempotent:** Yes | **Cacheable:** Yes

Request Body: SHOULD NOT send a body (no defined semantics; may be rejected as a request smuggling vector)

Typical Status Codes: 200 OK; 206 Partial Content (with Range header); 304 Not Modified (conditional GET)

- Primary mechanism of information retrieval; requests transfer of the current representation of the target resource.
- A client can issue a range request by including a Range header field, restricting the response to specified byte ranges.
- The response is cacheable; caches MAY use a stored GET response to satisfy later GET or HEAD requests.
- When URI parameters could encode sensitive data (e.g. form query strings), POST is preferred to avoid leaking data in URIs and server logs.
- GET and HEAD are the only methods all general-purpose HTTP servers MUST support.

HEAD (Section 9.3.2)

Safe: Yes | **Idempotent:** Yes | **Cacheable:** Yes

Request Body: SHOULD NOT send a body (same rationale as GET)

Typical Status Codes: Same header fields as the equivalent GET; no response body ever sent

- Identical to GET except the server MUST NOT send a body in the response.
- Used to retrieve representation metadata without the cost of transferring the full body; useful for cache validation and link checking.
- Server SHOULD send the same header fields it would send for the equivalent GET request; minor omissions (e.g. dynamically-generated Content-Length) are acceptable.
- A HEAD response is cacheable; it may also update or invalidate previously cached GET responses.

POST (Section 9.3.3)

Safe: No | **Idempotent:** No | **Cacheable:** Conditionally

Request Body: Yes — the enclosed representation is processed by the target resource according to its own semantics

Typical Status Codes: 201 Created; 200 OK or 204 No Content; 303 See Other; most 2xx/3xx/4xx/5xx are possible (except 206, 304, 416)

- Requests resource-specific processing of the enclosed body; semantics are defined by the target resource, not by the method.
- Common uses: submitting HTML form data, creating a new resource (server chooses the URI), appending to an existing representation, triggering server-side processing.
- If a new resource is created, the server SHOULD respond 201 Created with a Location header pointing to the new resource URI.
- NOT idempotent — clients SHOULD NOT auto-retry a failed POST unless they can verify the original was never applied.
- Cacheable only with explicit freshness directives plus a Content-Location identical to the POST target URI.
- Server MAY send 303 See Other to redirect to a cacheable GET representation of the result.

PUT (Section 9.3.4)

Safe: No | **Idempotent:** Yes | **Cacheable:** No

Request Body: Yes — the body defines the desired new state of the target resource

Typical Status Codes: 201 Created (resource did not previously exist); 200 OK or 204 No Content (successful replacement)

- Requests creation or replacement of the target resource state with the enclosed body. The client specifies the target URI (contrast with POST, where the server may choose it).
- Idempotent: repeating an identical PUT leaves the resource in the same state.
- Key distinction from POST: PUT's body replaces the target resource state; POST's body is processed by the resource's own semantics.
- Server SHOULD verify the representation is consistent with resource constraints; otherwise respond 409 Conflict or 415 Unsupported Media Type.
- Server MUST NOT send ETag or Last-Modified in a successful PUT response unless the data was stored without transformation.
- Origin servers SHOULD ignore unrecognised header and trailer fields in a PUT request — do not save them as resource state.

DELETE (Section 9.3.5)

Safe: No | **Idempotent:** Yes | **Cacheable:** No

Request Body: SHOULD NOT send a body (no defined semantics; may be rejected)

Typical Status Codes: 202 Accepted; 204 No Content; 200 OK

- Requests the server remove the association between the target resource and its current functionality — analogous to 'rm' in Unix: it deletes the URI mapping, not necessarily the underlying data.
- Idempotent: a second DELETE on the same URI has no additional intended effect (though the server may then return 404).
- Whether the underlying data is destroyed or archived is entirely up to the server implementation.
- Relatively few resources allow DELETE; its primary use is in remote authoring environments.
- Successful DELETE does not imply 200; 204 No Content is the most common success response.

CONNECT (Section 9.3.6)

Safe: No | **Idempotent:** No | **Cacheable:** No

Request Body: No — a CONNECT request message does not have content

Typical Status Codes: 2xx (tunnel established; connection switches to blind forwarding); any non-2xx means tunnel was not formed

- Requests the recipient (typically a proxy) establish a TCP tunnel to the host:port in the request target. Most commonly used for HTTPS proxying.
- Request target format is unique to this method: 'host:port' only — no scheme, no path. The port is ALWAYS required, even when it could be inferred.
- Server MUST reject CONNECT targeting an empty or invalid port, typically with 400 Bad Request.
- After a 2xx response, all data is forwarded blindly in both directions; the proxy stops interpreting HTTP on that connection.
- Significant security risk: proxies SHOULD restrict CONNECT to a limited set of known ports, preventing relay of non-HTTP protocols.
- Server MUST NOT send Transfer-Encoding or Content-Length in a 2xx CONNECT response; the client MUST ignore those fields if received.
- Intended for proxies; most origin servers do not implement CONNECT.

OPTIONS (Section 9.3.7)

Safe: Yes | **Idempotent:** Yes | **Cacheable:** No

Request Body: Optional — if content is sent, the client MUST include a valid Content-Type

Typical Status Codes: 200 OK with headers indicating supported options (e.g. Allow: GET, HEAD, POST)

- Requests information about the communication options available for the target resource without implying any resource action.
- Request target '*' (asterisk) applies to the server in general rather than a specific resource — useful as a capability probe or no-op 'ping'.
- A successful response SHOULD send any header indicating optional features applicable to the resource, notably Allow.
- Widely used as the CORS preflight mechanism (defined by the Fetch specification, which builds on OPTIONS).

TRACE (Section 9.3.8)

Safe: Yes | **Idempotent:** Yes | **Cacheable:** No

Request Body: MUST NOT send a body

Typical Status Codes: 200 OK whose body is the received request message (one representation is media type message/http)

- Requests a remote application-level loop-back: the final recipient reflects the received request back to the client as the response body.
- The final recipient is the origin server, or the first server to receive Max-Forwards: 0.
- Used for diagnostics: clients can see what a proxy or server actually received, notably the Via header which traces the request chain.
- Max-Forwards limits the depth of the request chain — useful for detecting proxies forwarding in an infinite loop.
- Clients MUST NOT include sensitive data (credentials, cookies) in a TRACE request, as it would be echoed back and could be disclosed.
- Often disabled in production as a precaution against Cross-Site Tracing (XST) attacks.

Additional Constraints and Requirements

405 Method Not Allowed — Allow Header is Mandatory

A 405 response MUST include an Allow header field listing the methods currently supported by the target resource (Section 10.2.1). This is a MUST, not a recommendation. Note that the set of allowed methods can change dynamically, so Allow reflects the state at response time.

501 Not Implemented vs. 405 Method Not Allowed

Use 501 when the origin server does not recognise or has not implemented the method at all. Use 405 when the method is recognised and implemented but is not allowed for that specific target resource. The Allow header is required for 405 but not for 501.

Cache Invalidation on Successful PUT and DELETE

A successful PUT or DELETE passing through a cache invalidates all stored responses for the target URI (see RFC 9111 Section 4.4). Without this, caches would serve stale representations after a resource is modified or removed.

Automatic Retry Rules

Clients MAY auto-retry idempotent requests (GET, HEAD, PUT, DELETE, OPTIONS, TRACE) after a connection failure. Clients SHOULD NOT auto-retry non-idempotent requests (POST, CONNECT) without evidence the original was never applied. Proxies MUST NOT auto-retry non-idempotent requests. A client SHOULD NOT auto-retry a failed automatic retry.

Request Content on Methods That Do Not Define It

GET, HEAD, and DELETE do not define semantics for request content. A client SHOULD NOT send a body on these methods; some implementations reject such requests and close the connection because of the request smuggling risk. CONNECT and TRACE go further: a CONNECT request message does not have content, and a client MUST NOT send content in a TRACE request.

CONNECT Responses — Forbidden Framing Headers

A server MUST NOT send Transfer-Encoding or Content-Length in a 2xx response to CONNECT, and a client MUST ignore those fields if received, because the connection immediately transitions to tunnel mode.

Method Token Case Sensitivity

HTTP method tokens are case-sensitive because they may act as a gateway to object systems with case-sensitive method names. 'GET' and 'get' are distinct tokens; implementations MUST NOT normalise method case.